

GSU Virtual Academic Advisor

An Agentic LLM System for Graduation Requirement Analysis

INF473 – Introduction to Generative AI

Galatasaray University, Department of Computer Engineering

Spring 2026

TL;DR. An agentic RAG system that reads a student’s transcript PDF and the official Galatasaray CE department ECTS regulations, then answers graduation questions grounded in both. `gpt-oss-120b` drives a tool-calling loop over a Qdrant vector store (semantically chunked by Gemini 2.5 Pro) and a locally parsed transcript. Everything observable via Langfuse, deployed on a Hugging Face Space, and re-exposed as a live MCP server. The system is end-to-end functional locally; the production endpoint is currently blocked by a network-side issue on Kloudeks’ egress allowlist (see Section 8).

This report is organised around the nine questions on the project brief. Each section is titled with the question it answers.

1 Why we picked this project

Almost every CE student has, at some point, opened the Bologna ECTS page, tried to work out whether they have enough technical electives, and given up. The official page is dense, internship rules (INF291, INF399) live in footnotes, and folklore like “*IA counts as a fail even though e-devlet shows credits*” is tribal knowledge passed between cohorts.

Beyond utility, the problem is a genuinely good fit for the modern GenAI stack: it is *retrieval-heavy*, it requires *cross-source reasoning* (rule book $\times$ personal transcript), and it benefits directly from *agentic tool use*. That is a far more honest test of the technology than yet another “summarise this PDF” demo.

A secondary motivation was **privacy**: transcripts are personal data, and we did not want them touching a US-hosted model. Routing the parser through Kloudeks (a Turkey-based inference provider) was a hard requirement, not a nice-to-have.

2 System architecture

The system has two pipelines that meet at the vector store (Figure 1).

Offline ingestion (runs once when rules change). The scraper (`src/ingestion/scrapper.py`) fetches the ECTS programme page and strips noise (scripts, navs, footers, cookie banners) using BeautifulSoup. The LLM chunker (`src/ingestion/chunker.py`) hands the cleaned HTML to Gemini 2.5 Pro under a strict Pydantic schema (`response_mime_type = application/json`, `response_schema = list[RuleChunk]`, `temperature = 0`), producing self-contained Turkish rule sentences tagged with topic and course code. The indexer (`src/ingestion/qdrant_push.py`) embeds each chunk with `qwen3-embedding-8b` (4096-d) and upserts to a Qdrant Cloud collection with cosine distance.

Online serving (every chat turn). The user lands on a Gradio UI (`app.py`) and optionally uploads a transcript. The file is MD5-hashed and cached in-process (capped at 50 entries to prevent leaks on the shared HF Space). The parser (`src/tools/transcript_parser.py`) extracts text with `pypdf` and asks `qwen3.5-35b` (Kloudeks) to emit a strict JSON: GPA, total credits, failed and passed

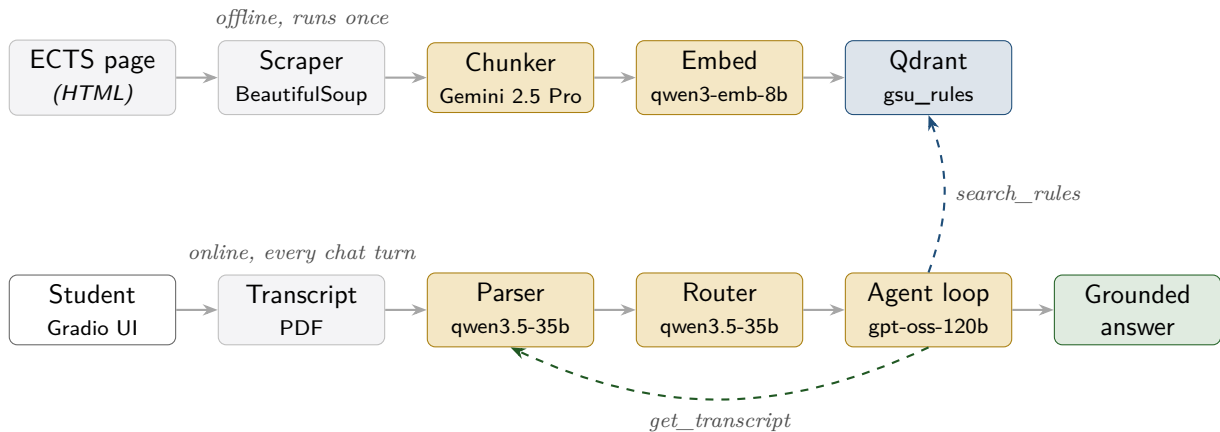


Figure 1: Two pipelines, meeting at the agent. Solid arrows: data flow. Dashed arrows: tool calls invoked dynamically by the agent at inference time.

courses. A router classifies the message as chit-chat (answered directly) or academic (delegated to the agent). The agent loop runs `gpt-oss-120b` with two tools: `search_graduation_rules` (Qdrant similarity, top- $k=10$, score threshold 0.45) and `get_student_transcript` (parsed JSON). The loop is hard-capped at 4 iterations.

Deployment. Gradio and a FastMCP server are mounted into the same FastAPI process on port 7860; the MCP endpoint is at `/mcp/sse`. Dockerised, CI/CD’d to a Hugging Face Space. Every router call, tool execution, and agent generation is wrapped with Langfuse’s `@observe`, with `session_id` propagated via `propagate_attributes` so each browser session is one coherent trace.

3 What is special about our design

Five things genuinely differentiate this from the standard “LangChain + Chroma + PDF loader” template:

1. **No LangChain.** We use the native OpenAI-compatible SDK and the new `google-genai` SDK directly. The whole codebase fits in under 1k lines of Python and the agent loop is roughly 40 lines. Tracing and debugging are dramatically cleaner.
2. **LLM-driven semantic chunking.** Rules in the ECTS page are scattered across `<td>` cells and tiny “Not 1, Not 2” footnotes. Naive splitters mangle them. Letting Gemini read the entire HTML and emit Pydantic-validated standalone sentences was the change that most visibly affected retrieval quality during development: before the switch the retriever was returning fragments of tables; after the switch it returned coherent rule sentences. We do not put a number on the magnitude of that change for the reasons set out in Section 7.
3. **Privacy-aware model routing.** Transcript parsing goes through Kloudeks (Turkey-hosted); the public rule base only touches Google Gemini, and only during offline ingestion. Personal data never leaves the country.
4. **Live MCP server, not just a chatbot.** The same tools the in-house agent uses are exposed via SSE at `/mcp/sse`, so any MCP-compatible client (Claude Desktop, Cursor, Claude Code) can query the GSU rule base remotely with a one-line config. This was not in the original spec — once the tools existed, adding it was almost free.
5. **Externalised prompts + hard memory ceilings.** Prompts live in `src/config/prompts.yaml` with Langfuse override, so we tune the agent without redeploying. The transcript cache is

MD5-keyed and capped at 50 entries to keep the deployed Space stable.

4 Algorithms used — and why

Layer	Choice	Why this and not something else
Embedding	qwen3-embedding-8b (4096-d)	The only embedding model the Kloudeks tenancy exposes to us. Because routing embeddings (and transcripts) through a Turkey-hosted provider is a hard privacy requirement, this was effectively a constraint, not a comparison <i>won</i> . We did not benchmark non-Kloudeks alternatives.
Vector DB	Qdrant Cloud, cosine	Managed; payload filtering by topic/rule type; modern <code>query_points</code> API with native score thresholds.
Chunker	Gemini 2.5 Pro, structured output	Long context window swallows the whole ECTS page; constrained decoding kills the “model wrapped JSON in markdown” bug class.
Parser/Router	qwen3.5-35b (Kloudeks)	Narrow JSON-emission tasks — the 120B model would be wasteful in both cost and latency. We did also try a Qwen <i>reasoning</i> model on this layer; see Section 7.
Agent	gpt-oss-120b (Kloudeks)	Reliable OpenAI-style tool calling; chains transcript × rule lookups correctly. The largest chat model the Kloudeks tenancy gave us.

Table 1: Per-layer model choices.

Why an agent loop, not single-shot RAG? Most non-trivial questions require *both* sources. “Do I have enough technical electives?” needs the rule (“*you need X ECTS*”) and the transcript (“*the student has Y*”). A single-shot retrieve-then-answer pipeline cannot decide which store to hit, let alone both. The agentic loop lets the model fetch what it actually needs, in the order it needs it, and we can see it doing so in the Langfuse trace.

5 The data

Source 1 — ECTS regulations. Scraped from <https://ects.gsu.edu.tr/tr/program/programmedetails/12>. The cleaned HTML is roughly 200 kB, which Gemini chunks into about 150 standalone Turkish rule sentences, each 30–100 words. Topics span first-semester compulsory courses, foreign language exemption, internship duration, technical-elective quotas, and graduation prerequisites. Every chunk carries `topic`, `related_course_code` (or `GENERAL`), and `rule_type`.

Source 2 — student transcripts (PDF, runtime). Two flavours in the wild: the Galatasaray-issued bureaucratic PDF and the e-devlet PDF. The latter has a known quirk — it reports credits even for failed (F / IA / NP) courses — which we explicitly correct for in both the parser prompt and a deterministic post-processing pass.

Schema after parsing. The transcript collapses to a compact JSON:

```
{
  "total_credits_completed": 192.0,
  "gpa": 3.12,
  "failed_courses": [{"code": "INF112", "grade": "F"}],
  "all_passed_courses": [{"code": "MAT101", "credits": 6.0}, ...]
}
```

Small enough to drop verbatim into any subsequent prompt context, which matters because the agent re-injects it when needed.

6 Related work in the literature

The general pattern — retrieval grounding for an LLM — comes from the original RAG paper (Lewis et al., 2020). For academic advising specifically, recent work and products group into three families:

- **Pure vector-DB RAG advisors** (the “CourseGPT”/AskUni/EduBot family). Split FAQ documents into ~ 500 -token chunks, embed, retrieve top- k , stuff into prompt. Adequate for “what is the add-drop deadline”-style questions; fails the moment the answer requires *your* data.
- **Structured curriculum planners** (DegreeWorks; many in-house tools at US universities). Every rule hand-coded in a rule engine; the transcript parsed by a fixed regex pipeline. Robust, but each new rule means shipping code.
- **Agentic systems with tool calling**. ReAct-style agents (Yao et al., 2023) plus OpenAI’s function-calling spec. The closest analogue to our work; public examples combine a SQL tool over a course catalogue with a vector tool over the policy text. Our contribution within this family is the combination of LLM-driven semantic chunking, privacy-aware model routing, and an MCP surface.

For Turkish embeddings the typical contenders are `multilingual-e5-large`, `BGE-M3`, and the `Qwen3` family. In practice the privacy constraint forced our hand: the Kloudeks tenancy only exposes one embedding model, so we did not compare these head-to-head — see Section 7.

7 Benchmarks and results

We did not run a benchmark. The Kloudeks tenancy available to us exposes exactly one embedding model (`qwen3-embedding-8b`) and two chat-capable models (`gpt-oss-120b` and `qwen3.5-35b`). Notably absent: any vision-language model. Our original plan for the transcript pipeline was to feed the rendered PDF pages directly to a VLM and skip `pypdf` entirely — transcript layouts are visually structured, and a VLM would have been the cleaner tool — but with no such model exposed on the tenancy, we stayed with text extraction. The privacy constraint that put us on Kloudeks in the first place rules out dropping in a non-Kloudeks alternative just to populate a comparison row. With one realistic option per layer, there is no like-for-like benchmark to run, so we are leaving the section descriptive rather than tabular.

One thing we did try, and can speak to. At one point we swapped the router and parser onto a Qwen *reasoning*-style model exposed via the same Kloudeks endpoint, on the assumption that “smarter” parsing would help. The opposite happened: end-to-end response time jumped by roughly **+600 s per turn**, because the reasoning model’s deliberation phase fired twice in sequence (once in the parser, once in the router) before the agent itself even started generating. We reverted both layers to `qwen3.5-35b`. It is a single data point and not a controlled experiment, but it is real, and it is the reason the architecture uses a non-reasoning model at the narrow-task layers.

What we can vouch for qualitatively. The system has been exercised end-to-end against both an e-devlet PDF and a Galatasaray-issued transcript. The agent reliably chains `search_graduation_rules` and `get_student_transcript` on questions that require both sources — this is observable, turn by turn, in the Langfuse trace. Latency on a warm turn sits around 4–7 s, dominated by the 120B generation step; a cold transcript upload adds 5–8 s for parsing. The score threshold of 0.45 (vs.

the default 0.0) was set by eye after watching low-score retrievals come back clearly off-topic — a hand-tuned default, not a formally-swept one.

8 What went wrong along the way

A non-exhaustive log, because being honest about the bumps is more useful than pretending it worked first try:

- **Naive chunking destroyed retrieval.** The first version used a recursive character splitter on raw HTML. Embeddings looked plausible in isolation but the chunks were fragments of tables. Replaced with the Gemini chunker.
- **Qdrant search deprecated mid-project.** We chased a flaky warning for an evening before migrating to `query_points` (commit 6818cb2).
- **Langfuse v3 → v4 broke tracing.** `propagate_attributes` has different semantics than the old `trace` helper. Rewired in a086404.
- **Tool-call hallucinations.** Early on, the agent would invoke `get_student_transcript` four times in a row, or call `search_graduation_rules` with an empty string. We added explicit “red line” prompt rules and a hard 4-iteration cap.
- **Markdown JSON fences.** The parser occasionally wrapped its output in `“`json. A re.DOTALL` extractor fishes the JSON body out as belt-and-braces.
- **Memory leak on the HF Space.** Without a cap, the transcript cache slowly ate Space RAM because each upload became a new tempdir path. MD5-keyed + 50-entry ceiling (19b8bb2) made the leak go away.
- **e-devlet transcripts lie about credits.** Failed (F / IA / NP) courses still show credits in the PDF. The parser used to count them; we now compensate in both the prompt and a deterministic post-processing step.
- **Empty queries spinning up a full loop.** Whitespace-only messages still kicked off the agent. Patched in b730e62.
- **Live deployment blocked by an IP-allowlist issue (network-side).** Locally the full pipeline works end-to-end without issues: parser, router, agent loop, Qdrant retrieval, MCP server, everything. On the Hugging Face Space, however, the Kloudeks endpoint consistently rejects our outbound requests — our best read is that the Space’s egress shows up as a non-allowlisted (effectively private/cloud) IP from Kloudeks’ perspective. The code, the prompts, the tools, the traces, the MCP surface — all demonstrable locally. The production endpoint is dark until network allowlisting is sorted on Kloudeks’ side; the fix lives outside our codebase.

9 What could be done next

- **Full graduation checklist view.** Deterministic, template-based render of every requirement and its status. The data is already there; this is purely a UI layer on top of existing tools.
- **Multi-department support.** The ingestion pipeline is generic; adding Industrial or Electrical Engineering is mostly a scraper URL and a chunk-level department label.
- **Cross-encoder re-ranker** (e.g. `bge-reranker-v2-m3`) on top-*k* retrieval — a standard next step for cutting the remaining off-topic hits we still see by eye.

- **An actual eval set.** A few-dozen queries reviewed with senior students, with ground-truth answers we can re-run on every change. We do not have this yet — which is exactly why Section 7 is honest about not having a benchmark to report.
- **Token streaming.** Kills the 4–7 s silence per turn at zero technical cost.
- **User accounts.** The cache is process-global and resets on redeploy; a tiny user store would skip re-uploads.
- **Course-recommendation tool.** “Which electives next semester get me to graduation fastest?” is a constraint-satisfaction problem on top of the same data — expose a planner as another agent tool.
- **Hardening the MCP surface.** Auth + per-client rate limits before any public production deploy.

Closing

What began as “a RAG chatbot for graduation rules” ended up teaching us considerably more about *where* the LLM stack tends to break than about LLMs themselves. Model choice mattered, but structural choices — semantic chunking, agentic tool calling instead of single-shot RAG, externalised prompts, and hard caps on both loops and caches — mattered more.

Source code: <https://github.com/hamzayakar/gsu-advisor> Live demo: <https://hamzayakar-gsu-advisor.hf.space> MCP endpoint: `/mcp/sse`